Mini-Prolog

Building a Logic Programming Interpreter in Rust

Implement a working interpreter for a subset of Prolog in Rust. Parse programs, unify terms, and evaluate queries using SLD-resolution with backtracking.

The domain anchor is a **family-tree database**. All example programs use kinship relations (`parent/2`, `ancestor/2`, `sibling/2`) to keep the logic concrete: your interpreter must correctly answer questions like "who are John's ancestors?" by searching the clause database.

The following program will serve as a running example throughout:

```
parent(john, mary).
parent(john, david).
parent(mary, alice).
parent(david, bob).

ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

The query `ancestor(john, Who).` should yield four answers: `Who = mary`, `Who = david`, `Who = alice`, `Who = bob`.

1 Learning Outcomes

Upon completion you will be able to:

ILO 1. **Define** a typed term algebra and clause representation suitable for a first-order logic programming language.

ILO 2. **Implement** Robinson's unification algorithm with occurs check, producing most general unifiers.

ILO 3. **Apply** SLD-resolution with backtracking to evaluate queries against a logic program, collecting all satisfying substitutions.

ILO 4. **Design** an extension to the resolution engine to support the cut operator, and **analyze** its effect on search behavior.

2 Self-Assessment Rubric

Use this to gauge your own understanding as you work.

ILO	Solid	Getting there	Needs work
1. Define term algebra	Types are precise and make invalid states unrepresentable. Clean `Display` impl.	Types are correct and complete. Minor issues.	Types are incomplete or poorly structured.
2. Implement unification	Correct MGU with occurs check. Composition is sound. All edge cases handled.	Unification works for common cases. Occurs check present but may miss edges.	Unification works only for trivial cases or is absent.
3. Apply SLD-resolution	All solutions found. Variable renaming is sound. REPL works.	Most solutions found. Backtracking may miss rare cases.	Only simple queries work. Backtracking incomplete.
4. Design/analyze cut	Cut is correctly implemented. Tests demonstrate understanding of its effect.	Cut is mostly correct. Minor deviations.	Cut is partially implemented or broken.

3 Part 1 — Terms, Clauses, and Parsing (10 pts)

Task 1.1: Define the Term type (3 pts)

Define a Rust `enum` representing Prolog terms with three variants:

- `Atom(String)` — lowercase identifiers: `john`, `parent`
- `Var(String)` — uppercase-starting identifiers: `X`, `Who`
- `Comp(String, Vec<Term>)` — compound terms: `parent(john, X)`

  Derive `Clone, Debug, PartialEq, Eq, Hash`.

Task 1.2: Define the Clause type (2 pts)

Define a `struct Clause` with fields `head: Term` and `body: Vec<Term>`. A fact is a clause with an empty body. Provide constructors `Clause::fact(head)` and `Clause::rule(head, body)`.

Task 1.3: Implement the parser (3 pts)

Implement a parser that accepts the following grammar:

```
program ::= clause*
clause  ::= term '.' | term ':-' terms '.'
terms   ::= term (',' term)*
term    ::= VAR | ATOM | ATOM '(' terms ')'
VAR     ::= [A-Z_][a-zA-Z0-9_]*
ATOM    ::= [a-z][a-zA-Z0-9_]*
```

The parser should return a `Vec<Clause>` (a `Program` type alias). You may use any parsing approach (hand-written recursive descent, `nom`, `pest`, etc.).

Task 1.4: Implement Display (2 pts)

Implement `std::fmt::Display` for `Term` and `Clause` so that they print as valid Prolog syntax. Verify round-tripping:

```
parse("parent(john, X).").to_string() == "parent(john, X)."
```

Checkpoint. Parse the family-tree program and print it back. You should recover the same program (modulo whitespace).

4 Part 2 — Unification (20 pts)

Unification is the heart of Prolog. You will implement Robinson's algorithm.

Task 2.1: Substitutions (4 pts)

Define a type for substitutions:

```
type Subst = HashMap<String, Term>;
```

Implement `fn apply(subst: &Subst, term: &Term) -> Term` that recursively replaces variables in a term according to the substitution map.

Task 2.2: Occurs check (2 pts)

Implement `fn occurs(var: &str, term: &Term) -> bool` that returns `true` if the variable named `var` appears anywhere in `term`.

Task 2.3: Robinson's unification (10 pts)

Implement:

```
fn unify(t1: &Term, t2: &Term, subst: &Subst) -> Option<Subst>
```

The algorithm proceeds by cases:

1. If both terms are the same variable, return the current substitution.
2. If one term is a variable `V`: apply the current substitution to both sides. If `V` is now bound, unify the binding with the other term. Otherwise, check that `V` does not occur in the other term (occurs check), then extend the substitution with `V` $\rightarrow$ `other term`.
3. If both are atoms with the same name, succeed.
4. If both are compound terms with the same functor and arity, unify arguments pairwise.
5. Otherwise, fail (return `None`).

Task 2.4: Substitution composition (4 pts)

Implement `fn compose(s1: Subst, s2: Subst) -> Subst` that composes two substitutions: the result applies `s1` then `s2`. Apply `s2` to every term in `s1` before merging.

Checkpoint. Verify:

- Unifying `f(X, a)` with `f(b, Y)` yields `{X→b, Y→a}`.
- Unifying `f(X, X)` with `f(a, b)` fails.
- Unifying `f(X)` with `f(g(X))` fails (occurs check).

5 Part 3 — SLD Resolution (20 pts)

With unification in hand, you can now execute queries.

Task 3.1: Variable renaming (3 pts)

Implement `fn rename_vars(clause: &Clause, suffix: usize) -> Clause` that produces a copy of the clause with every variable name suffixed with `_n` (where `n` is the suffix). This ensures clauses from the program have fresh variables each time they are used during resolution.

Task 3.2: SLD-resolution engine (12 pts)

Implement:

```
fn resolve(goals: &[Term], program: &[Clause], subst: Subst)
    -> Vec<Subst>
```

The algorithm:

1. If `goals` is empty, yield the current substitution.
2. Otherwise, take the first goal. For each clause in the program whose head unifies with the goal (after renaming the clause's variables):
   (a) Compute the MGU.
   (b) Apply it to the remaining goals.
   (c) Prepend the clause's body to the remaining goals.
   (d) Recurse with the new goals and the composed substitution.
3. Collect all successful substitutions into a vector and return it.

Task 3.3: Query interface (5 pts)

Build a minimal REPL or CLI that:

1. Loads a Prolog program from a file.
2. Accepts queries (terms ending with `?` or `.`).
3. For each query, calls `resolve` and prints each solution:

```
?- ancestor(john, Who).
Who = mary ;
Who = david ;
Who = alice ;
Who = bob.
false.
```

Checkpoint. Load the family-tree program. Query `ancestor(john, Who).` — you should get all four descendants. Query `ancestor(Who, alice).` — you should get `Who = mary` and `Who = john`.

6 Part 4 (Extension) — The Cut Operator (10 pts)

The cut operator `!` is Prolog's primary control mechanism. It commits to the current choice point, preventing backtracking past the point where the parent rule was selected.

Task 4.1: Parse the cut (1 pt)

Extend your parser to recognize `!` as a valid goal. Represent it as `Atom("!")` in your term type.

Task 4.2: Implement cut semantics (7 pts)

Modify `resolve` to handle cut. When a `!` goal is encountered:

1. It succeeds exactly once.
2. It prunes all remaining choice points created since the parent rule was entered — no further clauses are tried for the goal that selected this rule, and no further backtracking occurs past this point.

Hint: consider changing the return type to support early termination, or passing a flag/signal through the recursion to indicate that backtracking should stop. This will likely require **revisiting** the structure of your `resolve` function from Part 3.

Task 4.3: Demonstrate cut behavior (2 pts)

Write a test program that produces different results with and without cut:

```
test_cut(X) :- a(X), !, b(X).
a(1). a(2). a(3).
b(2). b(3).
```

The query `test_cut(X).` should yield *no results* (since `a(1)` commits via cut, but `b(1)` fails). Without the cut, it would yield `X = 2` and `X = 3`. Include this test case and explain the output.

7 Style

- Use `rustfmt` defaults.
- Use `Option`/`Result` idiomatically — no `unwrap()` outside of tests.
- Prefer small, composable functions over monolithic ones.
- Document public functions with `///` doc comments.
- Handle edge cases: empty programs, queries with no variables, self-referential terms.